



Høgskulen  
på Vestlandet

# DAT108 - Programmering og webapplikasjoner

## F01b – Repetisjon / Oppvarming Java

---

Lars-Petter Helland  
Bergen  
17. august 2026



# Repetisjon- / oppvarmingstemaer i dag

- › Interfaces (grensesnitt?, kontrakt?, ADT?), klasser, datatyper og objekter
  - › Indre klasser
  - › Anonyme (indre) klasser **NYTT**
- › Hvordan gi inn en fleksibel (polymorf) hjelpefunksjon til en metode
  - › Ikke nytt i seg selv, men å tenke på denne måten **~NYTT**

Hvis vi får tid:

- › Generics, typesikkerhet og typeparametre
- › Enhetstesting med JUnit



# Interfaces, klasser, datatyper og objekter

- › Hva er det norske ordet for **interface**?
  - › Grensesnitt? Kontrakt? ADT?
- › Hvilken konstruksjon kan være en variabel sin **(data)type**?
  - › Klasse? Interface? Primitiv datatype?
- › Hva vil det si at en klasse **implementerer** et interface?
- › Hvor(dan) defineres en klasse?
  - › I en egen fil der filnavn = klassenavn?
  - › Inni en annen klasse eller metode? (Indre / nøstet)
  - › **Uten klassenavn** (til engangsbruk) inni en annen klasse eller metode? (**Anonym indre**)
- › Datatype, referansevariabel og opprettelse av objekt?



# Interfaces, klasser, datatyper og objekter

- › Prosjekt: f01-interface-klasse-objekt
- › Pakke: `no.hv1.dat108.sirkelrektangel`
- › Fra scratch.
- › Legges ut i <https://github.com/lars-petter-helland/dat108-h2026> etter timen



# Eksempelet oppsummert

```
public interface Flate {  
    double areal();  
}
```

```
public class Rektangel implements Flate {  
    ...  
    @Override  
    public double areal() {  
        return bredde * hoyde;  
    }  
}
```

main:

```
Rektangel r = new Rektangel(3, 4);  
Flate f = new Rektangel(5, 6);  
System.out.println(r.areal());  
System.out.println(f.areal());
```

En klasse som “implements” et interface (en kontrakt) må inneholde implementasjoner av alle metodene listet opp i intefacet.

Referansevariabler til et objekt kan enten være av klassetypen (Rektangel) eller av en interfacetype som Rektangel implementerer (f.eks. Flate).

Vi vet da at objektet vi refererer til har metoden areal().



# Ulike måter å opprette klasser og objekter

- › Vi er vant til at vi definerer klasser egne filer. F.eks. vil vi definere klassen Person i filen Person.java.

- › Å opprette et Person-objekt gjør vi med

```
Person person = new Person(...)
```

- , og et et liste-objekt med

```
List<String> liste = new ArrayList<>()
```

- › Vi skal nå se på et par andre varianter å opprette klasser og objekter på som dere må beherske.



# Indre og anonyme klasser

- › Prosjekt: `f01-interface-klasse-objekt`
- › Pakke: `no.hv1.dat108.skapninghund`
- › Fra scratch.
- › Legges ut i <https://github.com/lars-petter-helland/dat108-h2026> etter timen



# Eksempelet oppsummert

```
public interface Skapning {
    void siHei();
}

main:

    class Fugl implements Skapning {
        @Override
        public void siHei() {
            System.out.println("Tweet!");
        }
    }

    Skapning fugl = new Fugl();
    fugl.siHei();

    Skapning katt = new Skapning() {
        @Override
        public void siHei() {
            System.out.println("Mjau!");
        }
    };
    katt.siHei();
```

Vi kan definere en klasse inni en annen klasse. Dette kalles en **indre klasse**. Vi har tidligere sett dette med Node-klassen i lenkede lister i DAT102.

Hvis vi definerer klassen inni en annen metode (her er Fugl definert inni main()) kalles det en lokal klasse.

Vi kan også opprette et objekt på direkten uten å oppgi noe navn på klassen. Dette kalles en **anonym (indre) klasse**.

katt-variabelen refererer her til et objekt av en (anonym) klasse som implementerer Skapning-interfacet. Vi vet derfor at katten kan siHei().



# Funksjoner, lambda-uttrykk og ...

Fra Canvas:

*Denne første modulen handler om konstruksjoner for funksjonell programmering i Java. Dere skal lære generelt om hva dette er, og dere skal kunne skrive programmer i Java som bruker lambda-uttrykk, funksjonelle kontrakter og strømmer.*

Vi begynner med et par enkle eksempler på hva vi snakker om. I Java er det litt kronglete (ser ikke skogen for bare trær), så jeg tenkte å vise i **Python** før vi går over til Java.



# Viser med Python først

Skriver eksemplene i en enkel tekst-editor (evt. VS Studio Code) i timen.

Python er ikke pensum i kurset, og poenget er ikke å få kjørt disse eksemplene.

De blir lagt ut i GitHub etterpå.



# Eksempelet oppsummert

Høyere ordens funksjoner i Python:

```
# En funksjon som lager en ny liste med f av  
alle verdiene i listen
```

```
def omformet_liste(liste, f):  
    ny_liste = []  
    for tall in liste:  
        ny_liste.append(f(tall))  
    return ny_liste
```

```
# En funksjon som kvadrerer et tall
```

```
def kvadrat(tall: int) -> int:  
    return tall * tall
```

```
nyListe = omformet_liste(listen, kvadrat)
```

Funksjonen `omformet_liste()` tar en annen funksjon `f()` som parameter.

Vi sier derfor at `omformet_liste()` er en høyere ordens funksjon.

Ved å gjøre det slik blir `omformet_liste()` mer generell og fleksibel, og kan gjenbrukes med ulike omformingsfunksjoner `f()`.



# Fleksibel hjelpefunksjon som parameter til metode

- › Som vi så i Python-eksemplet:
  - › Noen ganger kan det være praktisk å lage en generell metode, der samtidig ikke hvert eneste skritt i metoden er "hardkodet".
  - › F.eks. en metode som går gjennom alle verdier i en tabell og oppdaterer hver enkelt verdi etter en formel. Men i stedet for å hardkode oppdateringsfunksjonen, er det nyttig (mer fleksibelt) å kunne **gi funksjonen som en parameter til metoden**.
- › **Java** er et objekt-orientert språk som **ikke har direkte støtte for funksjoner** slik Python, JavaScript, Haskell m.fl. har.
- › **Hvordan gjøres dette da i Java?**



# Funksjoner i Java

- › Prosjekt: `f01-interface-klasse-objekt`
- › Pakke: `no.hv1.dat108.tallfunksjoner`
- › Fra scratch.
- › Legges ut i <https://github.com/lars-petter-helland/dat108-h2026> etter timen



# Eksempelet oppsummert

Føre på etter timen så langt vi kom



# Hvis vi får tid ...

men det får vi nok ikke 😊

(så da får vi ta det igjen senere)



# Generics, typesikkerhet og typeparametre

- › Hvorfor bruker vi **T** når vi kunne ha brukt **Object**?
  - › Typesikkerhet. Vi vet **hvilken datatype** elementene har slik at vi slipper å gjette.
  - › Typesikkerhet. Vi kan (fleksibelt) sikre at elementene er av **samme type**.
  - › Restriksjon. Vi kan sette **krav til elementtype** uten å miste fleksibilitet og typesikkerhet.
- › Vi kan bruke generiske typer når vi definerer klasser
- › Vi kan bruke generiske typer når vi definerer metoder



# Generics, typesikkerhet og typeparametre

- › Prosjekt: `f01-interface-klasse-objekt`
- › Pakke: `no.hv1.dat108.paravcomparables`
- › Legges ut i <https://github.com/lars-petter-helland/dat108-h2025> etter timen



# Enhetstesting med JUnit

- › Prosjekt: `f01-interface-klasse-objekt`
- › Pakke: `no.hv1.dat108.enhetstestingstabel`
- › Legges ut i <https://github.com/lars-petter-helland/dat108-h2025> etter timen



# Videre

- › Javahjelpen torsdag:
  - › Vi ser på helt grunnleggende ting i Java
- › Lekse til fredag:
  - › Jobb med "Oppgaver-uke34 - Repetisjonsoppgaver"
- › Forelesning fredag:
  - › Det vi skal jobbe med tilsv. **Video 1-12 fra Java 8 Lambda Basics**  
Sjekk gjerne ut!
  - › Før timen er det også greit at dere **kloner** mitt GitHub-repo, **puller** fra dette, og kopierer over aktuelle prosjekter i deres arbeidsmappe!
- › Oblig1: Blir lagt ut neste uke!

